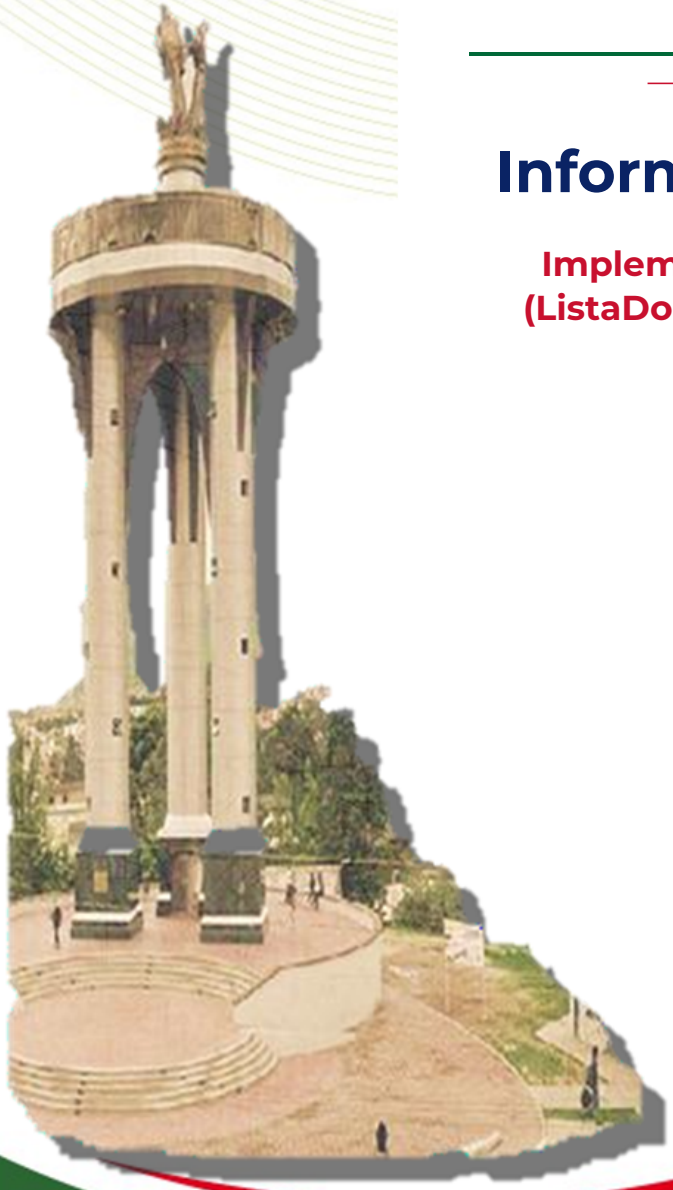
UNIVERSIDAD NACIONAL DE LOJA

FACULTAD DE LA ENERGÍA, LAS INDUSTRIAS Y LOS RECURSOS NATURALES NO RENOVABLES

CARRERA DE INGENIERÍA EN COMPUTACIÓN



Informe de Viabilidad Integral

Implementación de Navegación Bidireccional (ListaDoble), Pruebas Unitarias de Integración y Protocolo de Entrega EVA

Asignatura: Estructura de Datos

Grupo: 7

Integrantes:

Valeria Agila
Jose Valencia
Alison Tapia
Pilar Naranjo
Roy Gordillo

Fecha: 29/10/2026

1. Enlace al repositorio y evidencia Git

1.1. Enlace al repositorio

Repositorio del proyecto: `https://github.com/js-valencia/Trabajos_Estructura_De_Datos/tree/feature/avance4-lista-doble-grupo07`

1.2. Evidencia del Pull Request aprobado

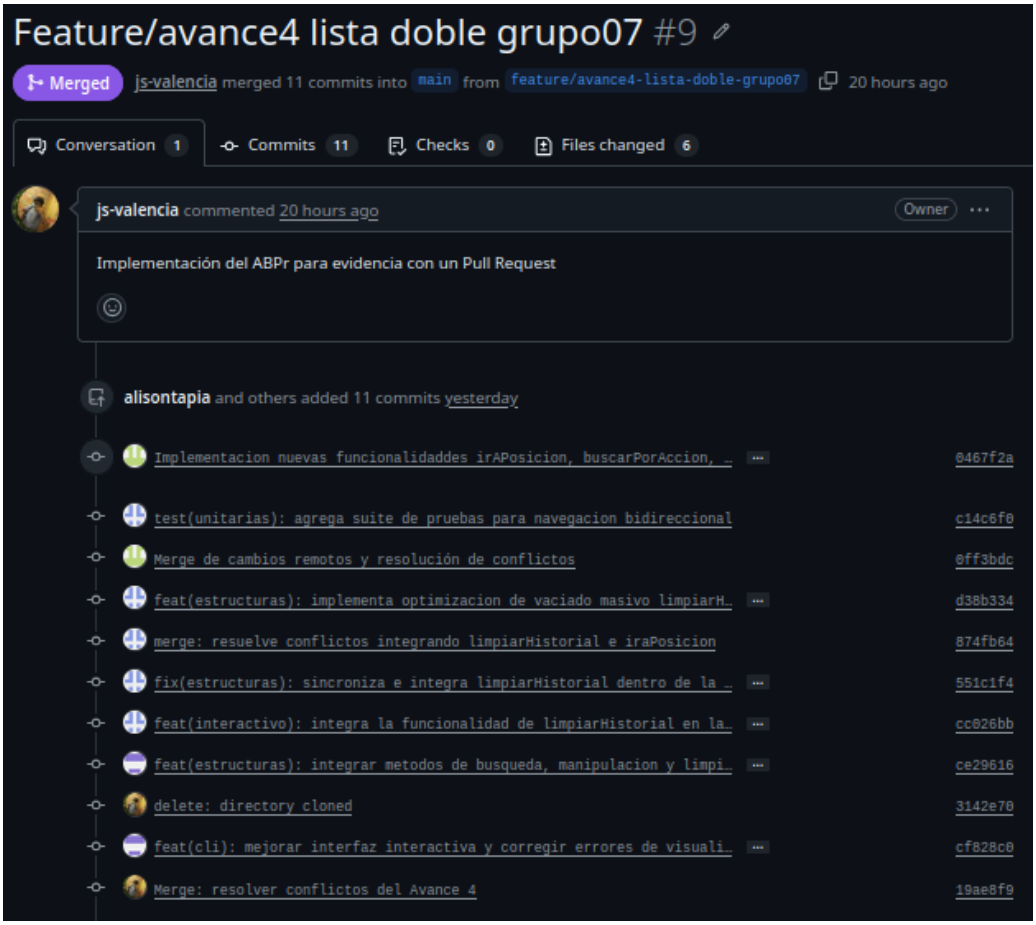


Figura 1: Pull Request aprobado.

1.3. Commit Hash del Merge a la rama principal

Hash del commit del merge: `40c99e5`

Enlace al commit: `https://github.com/js-valencia/Trabajos_Estructura_De_Datos/commit/40c99e5535a188a8999b0f1c37f38e179b5f8406`

2. Matriz de cobertura de pruebas

2.1. Pruebas unitarias implementadas

N.°	Prueba	Método evaluado	Resultado esperado	Estado	Tiempo (ms)
1	Inicialización	`insertar(dato)`, `obtenerActual()`	El cursor `actual` queda en el primer nodo insertado y `longitud` es 3.	OK	3,2827
2	Avance secuencial	`avanzar()`	Avanza por los nodos y devuelve `null` al llegar al final.	OK	0,5463
3	Retroceso secuencial	`retroceder()`	Retrocede por los nodos y devuelve `null` al llegar al inicio.	OK	2,3428
4	Eliminación de nodo intermedio	`eliminarActual()`	Reconecta `anterior` y `siguiente`, reduce `longitud` a 2 y mueve el cursor al siguiente.	OK	0,5166
5	Casos límite	`obtenerActual()`, `avanzar()`, `retroceder()`, `eliminarActual()`	En una lista vacía devuelven `null`, y `eliminarActual()` devuelve `false`.	OK	0,6474

Tabla 1: Resultados de la ejecución de `node --test`.

2.2. Resultados de ejecución de las pruebas

Se ejecutó la suite de pruebas unitarias desde la terminal mediante el comando `node --test`, que corresponde al ejecutor de pruebas integrado de Node.js [1].

Los resultados confirman que la navegación funciona en ambos sentidos y que los métodos `avanzar()` y `retroceder()` devuelven `null` al llegar a los extremos de la lista. También se comprobó que `eliminarActual()` reconecta correctamente los punteros `anterior` y `siguiente`, y que la estructura maneja de forma segura los casos límite, como una lista vacía.

Indicador	Valor
Pruebas ejecutadas (`tests`)	5
Aprobadas (`pass`)	5
Fallidas (`fail`)	0
Canceladas / omitidas (`cancelled` / `skipped`)	0 / 0
Duración total (`duration_ms`)	207,5806 ms

Tabla 2: Resumen de la ejecución.

2.3. Evidencia de las cinco pruebas en estado OK

La Figura 2 muestra la salida de la consola al ejecutar las pruebas unitarias, donde se observa que las cinco pruebas finalizaron correctamente, sin errores.

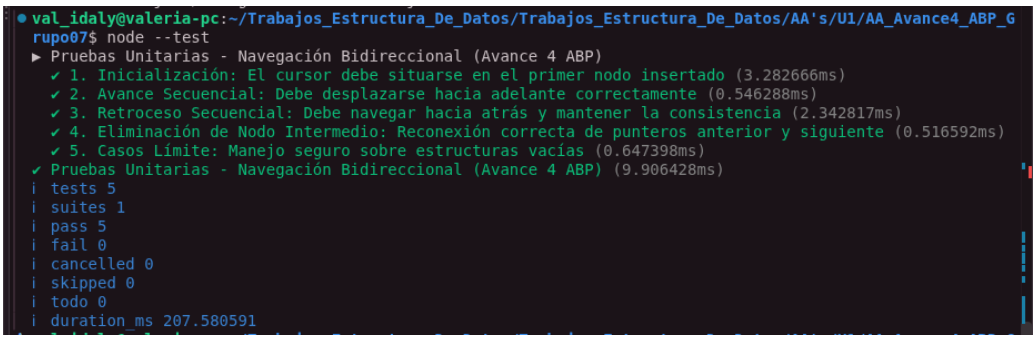


Figura 2: Ejecución de las cinco pruebas unitarias en estado OK.

3. Análisis de complejidad temporal y espacial

3.1. Complejidad temporal de `avanzar()`

El método `avanzar()` desplaza el cursor activo de la lista hacia el nodo siguiente:

```
avanzar() {
    if (this.actual && this.actual.siguiente) {
        this.actual = this.actual.siguiente;
        return this.actual.dato;
    }
    return null;
}
```

Complejidad temporal: $O(1)$ **(tiempo constante).**

El método ejecuta un número fijo de operaciones elementales, que no depende de la cantidad de nodos n almacenados en la lista: dos comprobaciones de existencia (`this.actual` y `this.actual.siguiente`), una asignación de referencia y un retorno del dato. No contiene ciclos ni recursión y no recorre la estructura, porque cada `NodoDoble` guarda un enlace directo a su vecino mediante el puntero `siguiente`. Por ello el costo es el mismo en el mejor, el peor y el caso promedio, incluyendo los casos límite (cursor en la cola o lista vacía), donde el método solo evalúa la condición y retorna `null`:

$$T(n) = c, \quad c \in \mathbb{R}^{+} \;\Rightarrow\; T(n) \in O(1).$$

Complejidad espacial: $O(1)$**.** El método no crea nodos ni estructuras auxiliares; únicamente reasigna la referencia `actual`, que ya existía. Tampoco genera referencias huérfanas, por lo que no añade trabajo al Recolector de Basura.

3.2. Complejidad temporal de `retroceder()`

El método `retroceder()` desplaza el cursor activo hacia el nodo anterior:

```
retroceder() {
    if (this.actual && this.actual.anterior) {
        this.actual = this.actual.anterior;
        return this.actual.dato;
    }
    return null;
}
```

Complejidad temporal: $O(1)$ **(tiempo constante).**

Es simétrico a `avanzar()`, pero utiliza el puntero `anterior`. Realiza dos comprobaciones, una asignación de referencia y un retorno, sin iteraciones ni recorridos, por lo que $T(n) \in O(1)$ en todos los casos.

La ventaja frente a una lista simplemente enlazada es clara: en esa estructura cada nodo solo conoce a su sucesor, de modo que retroceder exigiría recorrer la lista desde la cabeza hasta el nodo previo al actual, con un costo de $O(n)$. En `ListaDobleNavegacion`, cada nodo mantiene una referencia directa a su predecesor, lo que permite el desplazamiento hacia atrás en tiempo constante y justifica la transición a una lista doblemente enlazada en este avance [2].

Complejidad espacial: $O(1)$**.** Igual que en `avanzar()`, solo se reasigna un puntero existente y no se reserva memoria adicional.

Método	Complejidad temporal	Complejidad espacial
`avanzar()`	$O(1)$	$O(1)$
`retroceder()`	$O(1)$	$O(1)$

Tabla 3: Resumen comparativo de complejidades.

Observación sobre el resto de operaciones. Los métodos `irAlInicio()` e `irAlFinal()` también son $O(1)$, ya que la lista mantiene los punteros `cabeza` y `cola`. En cambio, `irAPosicion(k)` y `buscarPorAccion()` requieren recorrer nodos y son $O(n)$; `irAPosicion` recorre desde el extremo más cercano al índice solicitado, con a lo sumo $n/2$ saltos, pero su orden de crecimiento sigue siendo lineal. El costo en memoria de la estructura completa es $O(n)$, porque se almacena un nodo por elemento; el puntero adicional `anterior` solo añade una constante por nodo, y los métodos analizados no consumen espacio extra.

3.3. Complejidad temporal de `eliminarActual()`

El método elimina el nodo apuntado por el cursor `actual`. Como el cursor ya referencia a ese nodo, **no es necesario buscarlo**: el costo de localizarlo se pagó antes con `avanzar()`, `retroceder()` o `irAPosicion()`. El método ejecuta solo un número fijo de operaciones:

- Verificar si la lista está vacía (1 comparación).
- Reconectar los enlaces vecinos: `anterior.siguiente` y `siguiente.anterior`, o `cabeza` y `cola` cuando el nodo está en un extremo (hasta 2 asignaciones).
- Mover el cursor al nodo siguiente o, si no existe, al anterior (1 asignación).
- Anular `siguiente`, `anterior` y `dato` del nodo eliminado, y decrementar `longitud` (4 operaciones).

No hay ciclos ni recursión, y ninguna instrucción depende del tamaño n de la lista. Por tanto, el tiempo de ejecución es una constante c:

$$T(n) = c = \Theta(1).$$

Como el mejor y el peor caso coinciden, la complejidad es $\Theta(1)$. En una lista simplemente enlazada habría que recorrer la lista para hallar el predecesor, con costo $O(n)$; en la lista doble, el puntero `anterior` da acceso directo a él, lo que justifica la estructura.

Caso	Complejidad
Nodo intermedio	$O(1)$
Primer nodo (se actualiza `cabeza`)	$O(1)$
Último nodo (se actualiza `cola`)	$O(1)$
Único nodo / lista vacía	$O(1)$

Tabla 4: Análisis por casos de `eliminarActual()`.

3.4. Análisis de complejidad espacial

Cada `NodoDoble` almacena tres referencias: `dato`, `siguiente` y `anterior` [2]. Con el modelo de memoria del APE-01 (64 bits, palabra $w = 8$ B y cabecera de objeto $H = 3w = 24$ B):

$$S_{\text{nodo}} = H + 3w = 48 \text{ B}.$$

La lista guarda cuatro campos (`cabeza`, `cola`, `actual` y `longitud`), es decir, $S_{\text{lista}} = H + 4w = 56$ B. Para n elementos, siendo S_{dato} la memoria del objeto de dominio referenciado por `dato`:

$$S(n) = 56 + n\,(48 + S_{\text{dato}}) \text{ B} = \Theta(n).$$

Costo de la doble referencia. La lista doble usa un puntero extra por nodo respecto a la simple (+8 B por elemento). Es un factor constante que no altera el orden $\Theta(n)$, y a cambio se obtienen `retroceder()` y `eliminarActual()` en $O(1)$, `recorrerInverso()` sin reconstruir la lista (recorrido $O(n)$, con costo $O(1)$ por paso) e `irAPosicion()` con a lo sumo $n/2$ saltos, al partir desde el extremo más cercano.

Espacio auxiliar y liberación de memoria. `eliminarActual()` solo usa la variable local `nodoAEliminar` (una referencia de 8 B), por lo que su espacio auxiliar es $O(1)$. Además, tras reconectar los vecinos, el método asigna `null` a `siguiente`, `anterior` y `dato`. El nodo queda inalcanzable y el recolector de basura de V8 puede liberar sus $S_{\text{nodo}} + S_{\text{dato}}$ bytes, evitando fugas de memoria como las estudiadas en el AA-03.

4. Declaración de ética e inteligencia artificial

4.1. Herramientas de inteligencia artificial utilizadas

Se utilizó el asistente de inteligencia artificial **Claude** (Anthropic) como herramienta de apoyo para comprender los métodos del proyecto, revisar y corregir el formato en LaTeX del informe (carátula y tablas).

4.2. Prompts utilizados

1. **Prompt 1: complejidad de `eliminarActual()`.**

   "Necesito entender un método, no que me redactes ningún texto. Tengo este método de una lista doblemente enlazada en JavaScript:"

```
eliminarActual() {
    if (!this.actual) return false;
    const nodoAEliminar = this.actual;

    if (nodoAEliminar.anterior) {
        nodoAEliminar.anterior.siguiente = nodoAEliminar.siguiente;
    } else {
        this.cabeza = nodoAEliminar.siguiente;
    }

    if (nodoAEliminar.siguiente) {
        nodoAEliminar.siguiente.anterior = nodoAEliminar.anterior;
        this.actual = nodoAEliminar.siguiente;
    } else {
        this.cola = nodoAEliminar.anterior;
        this.actual = nodoAEliminar.anterior;
    }

    nodoAEliminar.siguiente = null;
    nodoAEliminar.anterior = null;
    nodoAEliminar.dato = null;
    this.longitud--;
    return true;
}
```

   "Explícame con palabras sencillas y un ejemplo: (1) qué hace cada línea; (2) por qué su complejidad temporal es $O(1)$ y qué tendría que pasar para que fuera $O(n)$; (3) cómo cambiaría si fuera una lista simplemente enlazada. Al final hazme dos preguntas para comprobar si lo entendí."

   Propósito: comprender el funcionamiento y la complejidad del método antes de redactar el análisis de la sección 3.3.

2. **Prompt 2: complejidad de `retroceder()`.**

   "Necesito entender otro método de la misma lista, no que me redactes ningún texto. Este es `retroceder()`:"

```
retroceder() {
    if (this.actual && this.actual.anterior) {
        this.actual = this.actual.anterior;
        return this.actual.dato;
    }
    return null;
```

```
}
```

"Explícame con palabras sencillas y un ejemplo: (1) qué hace cada línea y qué devuelve cuando el cursor está en el primer nodo o la lista está vacía; (2) por qué su complejidad temporal es $O(1)$ y qué tendría que pasar para que fuera $O(n)$; (3) por qué en una lista simplemente enlazada retroceder costaría $O(n)$. Al final hazme dos preguntas para comprobar si lo entendí."

Propósito: comprender el desplazamiento hacia atrás y su ventaja frente a una lista simplemente enlazada antes de redactar el análisis de la sección 3.2.

3. **Prompt 3: formato en LaTeX de la carátula y las tablas.**

   "Necesito corregir el formato de mi informe en LaTeX, no que me redactes el contenido. Quiero que la carátula quede en una sola hoja con el título y el subtítulo indicados, y que las tablas (matriz de cobertura de pruebas, resumen de ejecución y resumen de complejidades) tengan el mismo estilo de líneas, número y título, y que no se salgan del margen. Te paso mi código y el PDF generado; indícame qué cambiar y por qué."

   Propósito: corregir la presentación de la carátula y de las tablas del informe antes de la entrega, sin modificar el contenido ni el análisis de complejidad.

4.3. Declaración de autoría técnica

Los integrantes del grupo declaramos que el código y el análisis presentados en este informe son de nuestra autoría, y que las herramientas de inteligencia artificial se utilizaron únicamente como apoyo, conforme a lo indicado en esta sección.

5. Referencias

[1] OpenJS Foundation. «Test runner.» Node.js Documentation, visitado 31 de oct. de 2026. dirección: `https://nodejs.org/api/test.html`.

[2] C. A. Shaffer. «Doubly Linked Lists.» OpenDSA: Data Structures and Algorithms, Virginia Tech, visitado 31 de oct. de 2026. dirección: `https://tarjotin.cs.aalto.fi/cs-a1141/OpenDSA/Books/CS-A1141/html/ListDouble.html`.